

---

A Beginner's Study Guide

# Computer Vision, Neural Networks & Training YOLO11 Object Detectors

From the perceptron to a production-ready object detector: how neural networks learn, what makes CNNs and RNNs different, how YOLO turns an image into boxes and labels, and a complete walkthrough of every argument in Ultralytics' `model.train()` function — including data augmentation and the do's and don'ts of training for production.

---

|                         |                                                                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Prepared for</b>     | A computer vision beginner / student                                                                                                     |
| <b>Focus model</b>      | Ultralytics YOLO11 (with notes on the wider YOLO family)                                                                                 |
| <b>Covers</b>           | Neural networks • CNNs • RNNs • YOLO training • augmentation • production practice                                                       |
| <b>Reference source</b> | Ultralytics official documentation ( <a href="https://docs.ultralytics.com">docs.ultralytics.com</a> ), cross-checked at time of writing |

# Contents

---

## **Part 1 — Foundations — How a Model Actually Learns**

- 1.1 What machine learning training means
- 1.2 The neuron, weights, biases & activation functions
- 1.3 Forward pass, loss functions, backpropagation, gradient descent
- 1.4 Epochs, batches, learning rate & the training loop
- 1.5 Overfitting, underfitting & the train / val / test split

## **Part 2 — Neural Network Architectures: CNN & RNN**

- 2.1 Why plain neural networks struggle with images
- 2.2 Convolutional Neural Networks (CNNs): kernels, stride, padding, pooling
- 2.3 Recurrent Neural Networks (RNNs): sequences, hidden state, LSTM/GRU
- 2.4 CNN vs RNN vs Transformer — when each is used

## **Part 3 — Object Detection & the YOLO Family**

- 3.1 Classification vs. localization vs. detection vs. segmentation
- 3.2 How YOLO "sees" an image: grids, anchors, IoU, NMS, mAP
- 3.3 From YOLOv1 to YOLO11: what changed and why

## **Part 4 — Training YOLO11 Step by Step**

- 4.1 Installing Ultralytics & preparing a YOLO-format dataset
- 4.2 The training script and the `model.train()` call
- 4.3 Full parameter reference: core, optimizer, loss-weight settings
- 4.4 Data augmentation parameters, explained one by one
- 4.5 Reading the results: loss curves, mAP, confusion matrix

## **Part 5 — Training for Production: Do's and Don'ts**

- 5.1 Dataset quality & labeling
- 5.2 Splitting, leakage & evaluation honesty
- 5.3 Hyperparameters, monitoring & reproducibility
- 5.4 Export, deployment & ongoing monitoring

## **Part 6 — Study Path & Recommended Videos**

- 6.1 Suggested learning order
- 6.2 Video resources (3Blue1Brown and others)
- 6.3 Glossary of terms used in this guide

# Part 1

## Foundations — How a Model Actually Learns

### 1.1 What "training a model" means

A neural network is, underneath everything, a mathematical function with a huge number of adjustable numbers inside it — its **parameters** (weights and biases). When it is first created, those numbers are random, so the network's output is essentially noise. **Training** is the repeated process of: showing the network an example, checking how wrong its answer was, and nudging every parameter a tiny bit in the direction that would have made the answer less wrong. Do this millions of times over thousands of examples and the random function slowly turns into a function that maps images of cats to "cat" and images of cars to "car."

Everything else in this guide — CNNs, RNNs, YOLO, augmentation, optimizers — is really just detail on top of this one idea: *define how wrong the output is, then use calculus to figure out which direction reduces that wrongness, and move the parameters that way, repeatedly.*

### 1.2 The neuron, weights, biases & activation functions

The basic building block is the **artificial neuron**. It takes several numeric inputs ( $x_1, x_2, x_3 \dots$ ), multiplies each by its own learnable **weight** (how important that input is), adds them up, adds one more learnable number called the **bias** (a threshold/offset), and passes the result through a non-linear **activation function**:

```
output = activation( w1*x1 + w2*x2 + w3*x3 + ... + b )
```

A **layer** is simply many neurons processing the same inputs in parallel, each with its own weights. A **network** stacks layers one after another: the output of one layer becomes the input to the next. The first layer is the **input layer** (e.g. pixel values), the last is the **output layer** (e.g. class probabilities), and everything between is a **hidden layer**. A network with several hidden layers is what people mean by "deep" in deep learning.

The activation function matters enormously. Without it, stacking layers would be pointless — a stack of purely linear operations collapses back into a single linear operation, no matter how many layers you add. Non-linear activations are what let a network approximate the complicated, curved decision boundaries that real problems need.

| Function                     | Formula (intuition)             | Where you'll meet it                                                                                                                  |
|------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| ReLU (Rectified Linear Unit) | $\max(0, x)$                    | Default choice in most modern CNNs (including YOLO's backbone). Cheap to compute, avoids some training slowdowns older functions had. |
| Sigmoid                      | $1 / (1 + e^{-x})$              | Squashes any number into (0, 1). Used for binary outputs / objectness or confidence scores.                                           |
| Softmax                      | normalizes a vector to sum to 1 | Used on the final class-score layer so outputs can be read as class probabilities.                                                    |
| SiLU / Swish                 | $x * \text{sigmoid}(x)$         | Smooth ReLU variant used inside YOLO11's backbone and neck layers.                                                                    |

## 1.3 Forward pass, loss, backpropagation, gradient descent

### Forward pass

Feeding one example through the network from input to output, layer by layer, to get a prediction. This is also exactly what happens at inference/prediction time — training just adds the steps below on top of it.

### Loss function

A single number that measures how wrong the prediction was compared to the true label ("ground truth"). Training's entire goal is to make the **average loss over the dataset as small as possible**. Different tasks use different loss functions: cross-entropy for classification, mean-squared-error for regression, and for object detectors like YOLO, a combination of a box-regression loss (how well-placed the predicted box is), a classification loss (is the right class predicted) and a distribution-focal loss for box precision — more in Part 4.

### Backpropagation

Once we know the loss, we need to know how much each individual weight in the network contributed to it, so we can adjust it. Backpropagation is the algorithm that does this efficiently: it applies the calculus chain rule backward from the loss, layer by layer, computing the **gradient** (the slope, i.e. "if I increase this weight slightly, does the loss go up or down, and by how much") of the loss with respect to every single parameter in the network, in one organized backward sweep.

### Gradient descent

With every parameter's gradient in hand, gradient descent updates each parameter a small step in the direction that reduces the loss:

```
new_weight = old_weight - learning_rate * gradient
```

The **learning rate** controls the size of that step. Too large and training overshoots and never settles (or diverges); too small and training crawls and can get stuck. Most modern training doesn't use plain gradient descent but an **optimizer** built on top of it — SGD (with momentum, which keeps a running "velocity" so updates don't zig-zag) or adaptive optimizers like Adam/AdamW, which give each parameter its own effective learning rate. YOLO11 exposes this as the `optimizer` argument (Part 4).

## 1.4 Epochs, batches & the training loop

| Term             | Meaning                                                                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sample           | One labeled example — one image and its annotations.                                                                                                               |
| Batch            | A small group of samples processed together in one forward+backward pass, before the weights are updated once. Controlled by YOLO11's <code>batch</code> argument. |
| Iteration / step | One batch processed = one weight update.                                                                                                                           |

| Term  | Meaning                                                                                                                               |
|-------|---------------------------------------------------------------------------------------------------------------------------------------|
| Epoch | One full pass through the entire training dataset (every sample seen once). YOLO11 trains for a set number of epochs, by default 100. |

Why batches instead of one sample at a time (or the whole dataset at once)? A single sample gives a noisy, unrepresentative gradient; the whole dataset at once is often too large to fit in GPU memory and gives very few updates per epoch. A batch is the practical middle ground: enough samples to give a reasonably stable gradient estimate, small enough to fit in memory, and it lets you update the weights many times per epoch instead of just once.

Put together, one epoch of training looks like this:

```
for batch in training_data:
    predictions = model(batch.images)          # forward pass
    loss        = loss_function(predictions, batch.labels)
    gradients    = backpropagate(loss)         # backward pass
    optimizer.update(model.parameters, gradients) # gradient step
# after all batches: run on the validation set (no weight update)
# to see if the model is actually improving, not just memorizing
```

## 1.5 Overfitting, underfitting & the train / val / test split

A dataset is normally split into three parts, and this matters as much for YOLO as for any other model:

| Split            | Typical size                     | Purpose                                                                                                                                    |
|------------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Train            | ~70–85%                          | The data the weights are actually updated on.                                                                                              |
| Validation (val) | ~10–20%                          | Never used to update weights. Checked after every epoch to see genuine generalization, pick the best checkpoint, and drive early stopping. |
| Test             | ~5–15%, optional but recommended | Touched only once, at the very end, to report an honest, unbiased final performance number.                                                |

**Underfitting** is when the model is too simple, undertrained, or under-regularized to capture the pattern — training loss stays high. **Overfitting** is the opposite and far more common in practice: the model starts memorizing the specific training images (including their noise and quirks) instead of learning the general pattern. The telltale sign is training loss/accuracy keeps improving while validation loss/accuracy stalls or gets worse. YOLO11's patience argument automates a fix for this: it stops training if validation performance hasn't improved for N epochs ("early stopping").

**NOTE** — A model that scores 99% on the data it was trained on but 60% on new data has learned very little that's useful. The validation score, not the training score, is the number that tells you whether a model is actually good.

# Part 2

## Neural Network Architectures: CNN & RNN

### 2.1 Why plain ("fully connected") networks struggle with images

The simplest kind of network — every neuron in one layer connected to every neuron in the next, called a **fully connected** or **dense** network — treats its input as a flat list of numbers with no notion of spatial structure. Feed it a 224×224 color image and it sees  $224 \times 224 \times 3 \approx 150,000$  independent numbers. Two problems follow: first, the parameter count explodes (a single reasonably sized hidden layer would need hundreds of millions of weights, inviting severe overfitting); second, the network has no idea that a pixel and the pixel right next to it are related, or that a cat's ear looks the same whether it appears in the top-left or bottom-right of the photo. **Convolutional Neural Networks** were designed specifically to fix both problems.

### 2.2 Convolutional Neural Networks (CNNs)

Instead of connecting every input pixel to every neuron, a CNN slides a small learnable grid of numbers — a **kernel** or **filter** (commonly 3×3 or 5×5) — across the image. At every position it multiplies the kernel values with the pixels underneath and sums the result, producing one output value; sliding it across the whole image produces a 2-D grid of outputs called a **feature map**. Because the same small kernel is reused at every position, the parameter count stays tiny compared to a dense layer, and a pattern the kernel learns to detect (an edge, a corner, a patch of fur texture) is recognized wherever it appears in the image — a property called **translation invariance**.

#### Key CNN vocabulary

| Term                  | What it means                                                                                                                                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kernel / filter       | The small matrix of learnable weights that slides over the image. A layer usually learns many kernels in parallel, each detecting a different pattern (edges, colors, textures).                                |
| Stride                | How many pixels the kernel moves each step. Stride 1 moves one pixel at a time (large, detailed output); stride 2 skips every other position (halves the output size, common in YOLO's backbone to downsample). |
| Padding               | Adding a border of zeros around the image so the kernel can also center on edge pixels, and so the output size can be controlled (e.g. kept the same as the input).                                             |
| Feature map / channel | The output of one kernel swept across the image. A convolutional layer with 64 kernels produces 64 feature maps ("64 channels").                                                                                |
| Pooling (max/avg)     | A downsampling step that shrinks feature maps by keeping only the maximum (or average) value in each small region, reducing computation and adding a small amount of positional tolerance.                      |
| Receptive field       | How much of the original image a single value in a deep feature map "sees." It grows with every convolution/pooling layer, which is why deep layers can recognize whole objects, not just edges.                |

A typical CNN stacks many convolution+activation+(sometimes pooling) blocks. Early layers learn simple things — edges, colors, gradients. Middle layers combine those into textures and parts (an eye, a wheel, a corner of a

window). Deep layers combine parts into whole objects (a face, a car). This backbone of stacked convolutions is exactly what sits underneath YOLO, ResNet, EfficientNet and almost every modern vision model — YOLO11's backbone is a CNN, followed by a "neck" that merges features from different depths, and a "head" that turns the final feature maps into box coordinates and class scores (Part 3).

### Watch

#### ■ But what is a convolution? — 3Blue1Brown

<https://www.youtube.com/watch?v=KuXjwB4LzSA>

Visual, intuitive walkthrough of what the sliding-kernel operation is actually doing, using image blurring/edge-detection as the running example.

## 2.3 Recurrent Neural Networks (RNNs)

CNNs are built for data with spatial structure (images). **Recurrent Neural Networks** are built for data with **sequential/temporal structure** — text, audio, sensor readings over time, video frame sequences. The core idea: instead of processing an entire input at once, an RNN processes it one step at a time (one word, one time-sample, one frame) and keeps a running internal summary called the **hidden state**, which is updated at every step and carries information forward from everything seen so far.

```
hidden_state = 0
for item in sequence:
    hidden_state = f(item, hidden_state, weights)  # same weights reused every step
    output       = g(hidden_state)
```

Plain RNNs have a well-known weakness: over long sequences, gradients flowing backward through many repeated steps tend to shrink toward zero (the **vanishing gradient problem**) or blow up, so the network struggles to connect information that is far apart in the sequence. Two architectures were designed to fix this by adding learnable "gates" that control what information is kept, forgotten, or passed on:

| Variant                       | Idea                                                                                                                                                      |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| LSTM (Long Short-Term Memory) | Adds a separate "cell state" plus input/forget/output gates that learn what to remember and what to discard. The classic fix for long-range dependencies. |
| GRU (Gated Recurrent Unit)    | A simplified LSTM with fewer gates (update + reset). Usually trains faster, with similar performance on many tasks.                                       |

**NOTE** — YOLO11 itself is a pure CNN-based detector and does not use RNNs — it processes each image independently. RNNs (and their modern successor, the Transformer's attention mechanism) matter if you later work on tasks with a time/sequence axis: multi-object tracking across video frames, video action recognition, or captioning. They're included here because they're one of the three architecture families (alongside plain feed-forward networks and CNNs) every CV/ML beginner is expected to know.

## 2.4 CNN vs. RNN vs. Transformer — quick comparison

|                                   | CNN                                     | RNN                                             | Transformer                                                           |
|-----------------------------------|-----------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------|
| Best for                          | Spatial data: images, feature grids     | Sequential data: text, time series, audio       | Both, at larger scale: language, and increasingly vision (ViT)        |
| Core mechanism                    | Sliding shared kernels (local patterns) | Step-by-step hidden state (memory)              | Self-attention: every element looks at every other element directly   |
| Processes input                   | All at once (parallel)                  | One step at a time (sequential)                 | All at once (parallel)                                                |
| Where you'll see it in this guide | YOLO11's backbone/neck/head             | Not used by YOLO11; relevant for video/tracking | Used by some newer detectors (e.g. RT-DETR) as an alternative to YOLO |

### If any of this felt fast

#### ■ But what is a neural network? — 3Blue1Brown (Deep Learning, Chapter 1)

<https://www.youtube.com/watch?v=aircAruvnKk>

Best starting point for the whole topic; builds the neuron/layer/activation picture visually using handwritten-digit recognition.

# Part 3

## Object Detection & the YOLO Family

### 3.1 Classification vs. localization vs. detection vs. segmentation

| Task             | Question it answers                             | Output                                                               |
|------------------|-------------------------------------------------|----------------------------------------------------------------------|
| Classification   | "What is the main thing in this image?"         | One label per image ("cat")                                          |
| Localization     | "Where is the main thing?"                      | One label + one bounding box                                         |
| Object detection | "What objects are here, and where is each one?" | Many labels, each with its own bounding box (this is what YOLO does) |
| Segmentation     | "Which exact pixels belong to each object?"     | A pixel-level mask per object, not just a rectangle                  |

YOLO11 supports several of these tasks through one framework (detect, segment, classify, pose-estimation, oriented bounding boxes), but its headline task — and the focus of this guide — is **object detection**.

### 3.2 How YOLO "sees" an image

"YOLO" stands for **You Only Look Once**. The name refers to its founding idea (2015, Joseph Redmon): older detectors ran a classifier over hundreds or thousands of candidate regions in an image separately ("two-stage" detectors), which was accurate but slow. YOLO instead treats detection as a single regression problem: one forward pass of one CNN over the whole image directly outputs all the bounding boxes, their classes, and confidence scores at once. That's what makes it fast enough for real-time video.

#### The core building blocks

| Concept                       | What it means                                                                                                                                                                                                                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Grid cells                    | The image is conceptually divided into a grid; each cell is responsible for detecting objects whose center falls inside it.                                                                                                                                                              |
| Bounding box                  | A predicted rectangle around an object, usually stored as a center (x, y), width and height, all relative to the image.                                                                                                                                                                  |
| Confidence / objectness score | How sure the model is that a given box actually contains an object at all (separate from which class it is).                                                                                                                                                                             |
| Class score                   | Given that there is an object, the probability distribution over the possible classes ("dog" 0.9, "cat" 0.05, ...).                                                                                                                                                                      |
| IoU (Intersection over Union) | The standard way to measure how well two boxes overlap: $\text{area of overlap} \div \text{area of union}$ . $\text{IoU} = 1$ means a perfect match; $\text{IoU} = 0$ means no overlap. Used both to decide if a prediction "counts" as correct during evaluation, and inside NMS below. |
| NMS (Non-Maximum Suppression) | A detector typically produces several overlapping boxes for the same object. NMS keeps the highest-confidence box and removes other boxes that overlap it above an IoU threshold, leaving one box per object.                                                                            |

| Concept                      | What it means                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mAP (mean Average Precision) | The standard accuracy metric for detectors. It summarizes precision (how many predicted boxes were correct) against recall (how many real objects were found) across all classes and confidence thresholds. "mAP50" uses an IoU threshold of 0.5 to decide correctness; "mAP50-95" averages this over stricter thresholds from 0.5 to 0.95, a harder and more informative number. |

**NOTE** — Modern YOLO versions (including YOLO11's default head) are largely "anchor-free": rather than starting from a fixed set of pre-defined box shapes ("anchors") at each grid cell and adjusting them, the model directly predicts box boundaries and uses a distribution-based regression (Distribution Focal Loss) for more precise, flexible edges. You don't need to configure this — it's mentioned only so the terms "anchor-free" and "DFL" make sense when you see them in loss argument names (dfl, Part 4).

### 3.3 From YOLOv1 to YOLO11: what changed and why

| Version                     | Introduced by                                   | What it added                                                                                                                                                                                                                                   |
|-----------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YOLOv1 (2015)               | Redmon et al.                                   | Introduced the single-pass, grid-based detection idea. Fast, but weak on small/overlapping objects.                                                                                                                                             |
| YOLOv2 / YOLO9000 (2016)    | Redmon & Farhadi                                | Added anchor boxes, batch normalization, could detect 9,000+ categories via a joint dataset trick.                                                                                                                                              |
| YOLOv3 (2018)               | Redmon & Farhadi                                | Multi-scale predictions (detects small, medium and large objects from different feature-map depths), a deeper backbone (Darknet-53).                                                                                                            |
| YOLOv4 / YOLOv5 (2020)      | Bochkovskiy et al. / Ultralytics                | Modernized training tricks (mosaic augmentation, CSP blocks); YOLOv5 was the first fully PyTorch, pip-installable Ultralytics implementation, hugely improving ease of use.                                                                     |
| YOLOv6 – YOLOv9 (2022–2024) | Various (Meituan, Ultralytics, academic groups) | Iterative architecture/training refinements: re-parameterized blocks, better label assignment, programmable gradient information (v9).                                                                                                          |
| YOLOv10 (2024)              | Tsinghua Univ.                                  | NMS-free training for lower end-to-end latency.                                                                                                                                                                                                 |
| YOLO11 (2024)               | Ultralytics                                     | The version this guide targets: refined backbone/neck (C3k2, C2PSA blocks) for a better accuracy/speed trade-off at a similar parameter count to YOLOv8, plus first-class support for detect, segment, classify, pose and OBB tasks in one API. |
| YOLO12 / YOLO26 (2025–2026) | Ultralytics & others                            | Newer releases exist beyond YOLO11 at the time of writing; the training workflow and most arguments described in Part 4 carry over almost unchanged, since they belong to the shared Ultralytics training engine.                               |

The reason this guide (and most production teams in 2026) still centers on YOLO11 specifically is that it sits in a mature, well-documented, well-supported sweet spot: excellent accuracy-per-FLOP, broad export support (ONNX, TensorRT, OpenVINO, CoreML, TFLite, and more), and the same simple `model.train()` API this guide walks through in Part 4.

# Part 4

## Training YOLO11 Step by Step

---

### 4.1 Installing Ultralytics & preparing a dataset

Install

```
pip install ultralytics
```

YOLO expects a dataset in a specific folder layout and a small YAML file describing it. Each image has a matching .txt label file with one line per object: `class_id x_center y_center width height`, all coordinates normalized to 0–1 relative to the image size.

Folder layout

```
dataset/  
  images/  
    train/img001.jpg  
    val/img101.jpg  
  labels/  
    train/img001.txt  
    val/img101.txt
```

Dataset YAML

```
# data.yaml  
path: /absolute/or/relative/path/to/dataset  
train: images/train  
val: images/val  
names:  
  0: person  
  1: forklift  
  2: hard_hat
```

**NOTE** — Tools like Roboflow, CVAT, or Label Studio can label images and export directly in this YOLO format, which saves you from writing the .txt files by hand.

### 4.2 The training script

Minimal training script

```

from ultralytics import YOLO

# Load a pretrained checkpoint (recommended) or build from YAML for scratch training
model = YOLO("yolo11n.pt")    # n = nano; s, m, l, x get progressively larger/slower/more
                              # accurate

results = model.train(
    data="data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    device=0,          # GPU 0; use "cpu" if no GPU
    project="runs",
    name="forklift-detector",
)

```

Two loading choices matter: `YOLO("yolo11n.pt")` loads **pretrained COCO weights** and fine-tunes them on your data — almost always the right choice, since the backbone already knows general visual features (edges, textures, shapes) and only needs to adapt to your specific classes. `YOLO("yolo11n.yaml")` instead builds the architecture with random weights and trains fully from scratch — only worth doing with very large datasets (100,000+ images) or a genuinely novel domain (e.g. X-ray or hyperspectral imagery) where COCO's natural-image features don't transfer well.

Model sizes, from smallest/fastest to largest/most accurate: `yolo11n` (nano) → `yolo11s` (small) → `yolo11m` (medium) → `yolo11l` (large) → `yolo11x` (extra-large). Start with nano or small to iterate quickly; move up only if accuracy plateaus below what you need and your deployment hardware can afford the extra latency.

## 4.3 Full parameter reference for `model.train()`

Every argument below can be passed as a keyword to `model.train(...)` or as `arg=value` on the CLI (`yolo detect train arg=value ...`). Defaults shown are Ultralytics' official defaults at the time of writing; augmentation arguments are covered separately in 4.4.

### Core settings — what to train, how long, on what hardware

| Argument              | Type      | Default | What it does                                                                                                                        |
|-----------------------|-----------|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>model</code>    | str       | None    | Path to a .pt pretrained checkpoint or a .yaml architecture file. Defines the network to train.                                     |
| <code>data</code>     | str       | None    | Path to the dataset YAML (train/val paths + class names).                                                                           |
| <code>epochs</code>   | int       | 100     | Total passes over the full training set.                                                                                            |
| <code>time</code>     | float     | None    | Maximum training time in hours; overrides epochs if set. Useful for time-boxed training jobs.                                       |
| <code>patience</code> | int       | 100     | Epochs to wait with no validation-metric improvement before early stopping.                                                         |
| <code>batch</code>    | int/float | 16      | Images per batch. <code>batch=-1</code> auto-sizes for ~60% GPU memory; <code>batch=0.70</code> targets a specific memory fraction. |
| <code>imgsz</code>    | int       | 640     | Training image size (images are resized to this, roughly square).                                                                   |
| <code>save</code>     | bool      | True    | Save training checkpoints and final weights.                                                                                        |

| Argument      | Type         | Default | What it does                                                                                                                                    |
|---------------|--------------|---------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| save_period   | int          | -1      | Save a checkpoint every N epochs in addition to the best/last; -1 disables.                                                                     |
| cache         | bool/str     | False   | Cache images in RAM (True/'ram') or on disk ('disk') to speed up data loading at the cost of memory/disk space.                                 |
| device        | int/str/list | None    | Compute device: 0 (GPU 0), [0,1] (multi-GPU), 'cpu', 'mps' (Apple Silicon), 'npu:0' (Huawei Ascend).                                            |
| workers       | int          | 8       | Background threads loading/augmenting data per GPU process.                                                                                     |
| project       | str          | None    | Root output folder for runs.                                                                                                                    |
| name          | str          | None    | Sub-folder name for this specific run, under project.                                                                                           |
| exist_ok      | bool         | False   | If True, allows overwriting an existing project/name folder instead of auto-incrementing it.                                                    |
| pretrained    | bool/str     | True    | Start from pretrained weights (True), a specific weights path, or random init (False, architecture only).                                       |
| resume        | bool         | False   | Continue an interrupted run from its last checkpoint, restoring weights, optimizer state and epoch count.                                       |
| seed          | int          | 0       | Random seed, for reproducible runs.                                                                                                             |
| deterministic | bool         | True    | Forces deterministic algorithms for reproducibility (may cost some speed).                                                                      |
| single_cls    | bool         | False   | Treats every class as one class - useful for pure "is there an object here" detectors.                                                          |
| classes       | list[int]    | None    | Restrict training to a subset of class IDs from the dataset.                                                                                    |
| rect          | bool         | False   | Minimal padding per batch (rectangular training) instead of square resizing; can speed up training slightly.                                    |
| multi_scale   | float        | 0.0     | Randomly varies imgsz per batch by +/- this fraction, helping the model generalize across object scales.                                        |
| amp           | bool/str     | True    | Automatic Mixed Precision (FP16/BF16) training - faster and lighter on GPU memory with minimal accuracy impact on supported GPUs.               |
| fraction      | float        | 1.0     | Train on a subset of the dataset (e.g. 0.1 = 10%) - handy for quick pipeline smoke-tests.                                                       |
| freeze        | int/list     | None    | Freezes the first N layers (or named layers) so they are not updated - common in transfer learning to keep pretrained backbone features intact. |
| plots         | bool         | True    | Save training/validation plots (loss curves, PR curve, confusion matrix, prediction samples).                                                   |
| val           | bool         | True    | Run validation after each epoch.                                                                                                                |
| verbose       | bool         | True    | Print per-epoch progress and metrics to the console.                                                                                            |

## Optimizer, learning-rate schedule & warmup

| Argument        | Type  | Default | What it does                                                                                                                                      |
|-----------------|-------|---------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| optimizer       | str   | 'auto'  | SGD, Adam, AdamW, NAdam, RAdam, RMSProp, or 'auto' (picks AdamW/MuSGD based on training length). Affects convergence speed/stability.             |
| lr0             | float | 0.01    | Initial learning rate (typical: 1e-2 for SGD, 1e-3 for Adam-family). The single most impactful hyperparameter to tune.                            |
| lrf             | float | 0.01    | Final learning rate as a fraction of lr0 (i.e. the rate decays down to lr0*lrf by the end of training).                                           |
| cos_lr          | bool  | False   | Use a cosine-shaped decay curve for the learning rate instead of the default linear decay - often gives smoother convergence.                     |
| momentum        | float | 0.937   | SGD momentum / Adam beta1 - how much past gradients influence the current update, smoothing out noisy steps.                                      |
| weight_decay    | float | 0.0005  | L2 regularization strength; penalizes large weights to reduce overfitting.                                                                        |
| warmup_epochs   | float | 3.0     | Epochs spent ramping the learning rate up from a low value to lr0, which stabilizes the very start of training when gradients are least reliable. |
| warmup_momentum | float | 0.8     | Momentum value used during the warmup period.                                                                                                     |
| warmup_bias_lr  | float | 0.1     | Separate (typically higher) learning rate for bias parameters during warmup.                                                                      |
| close_mosaic    | int   | 10      | Disables mosaic augmentation for the last N epochs, letting the model fine-tune on realistic, unmodified image statistics before training ends.   |

**NOTE** — If you only tune one thing, tune lr0. Too high: loss oscillates or explodes to NaN. Too low: training is painfully slow and may settle in a poor local minimum. A reasonable starting move is to try the default first, then a 3-10x smaller value if loss is unstable, on a short trial run before committing to a full training budget.

## Loss-component weights

YOLO's total loss is a weighted sum of several sub-losses. These weights let you shift emphasis - e.g. raise box if your boxes are well-classified but poorly localized.

| Argument | Type  | Default | What it does                                                                                                                                     |
|----------|-------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| box      | float | 7.5     | Weight of the bounding-box regression loss (how precisely boxes are placed).                                                                     |
| cls      | float | 0.5     | Weight of the classification loss (is the predicted class correct).                                                                              |
| dfl      | float | 1.5     | Weight of the Distribution Focal Loss term, which refines box-edge precision under the anchor-free head.                                         |
| cls_pw   | float | 0.0     | Power for class re-weighting to counter class imbalance from inverse class frequency; 0 disables it, 1 applies full inverse-frequency weighting. |
| nbs      | int   | 64      | Nominal batch size used to normalize loss magnitude so results are comparable across different actual batch sizes.                               |

## Logging, checkpoints & validation details

| Argument | Type     | Default | What it does                                                                                                |
|----------|----------|---------|-------------------------------------------------------------------------------------------------------------|
| save_dir | str      | None    | Exact output directory, overriding the project/name auto-naming.                                            |
| profile  | bool     | False   | Profiles ONNX/TensorRT inference speed during training - useful when optimizing for deployment.             |
| dropout  | float    | 0.0     | Dropout rate for classification-task heads, randomly disabling units during training to reduce overfitting. |
| max_det  | int      | 300     | Maximum detections kept per image during training-time validation.                                          |
| compile  | bool/str | False   | Enables PyTorch 2.x torch.compile graph compilation for potential speed-ups on supported setups.            |

**NOTE** — This table lists the arguments a beginner or intermediate user will actually touch. The Ultralytics configuration page ([docs.ultralytics.com/usage/cfg](https://docs.ultralytics.com/usage/cfg)) documents additional task-specific arguments - pose/OBB/segmentation loss weights, Solutions settings, export options - which fall outside plain object detection.

## 4.4 Data augmentation, explained one by one

Augmentation applies random, label-preserving transformations to each training image on the fly, so the model effectively sees many more variations than the raw dataset contains. This is one of the biggest levers against overfitting, especially on small datasets, because it teaches the model that an object is still the same object under different lighting, angle, position and scale.

| Argument     | Default           | What it does / when to change it                                                                                                                                                                                                 |
|--------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hsv_h        | 0.015             | Randomly shifts image hue (color) by this fraction of the color wheel - builds tolerance to different lighting/white-balance.                                                                                                    |
| hsv_s        | 0.7               | Randomly shifts color saturation - simulates washed-out or vivid lighting conditions.                                                                                                                                            |
| hsv_v        | 0.4               | Randomly shifts brightness (value) - simulates darker/brighter scenes.                                                                                                                                                           |
| degrees      | 0                 | Random rotation range in degrees. Raise this if your real-world objects can appear at any angle (e.g. aerial imagery); keep near 0 for naturally upright scenes.                                                                 |
| translate    | 0.1               | Random horizontal/vertical shift as a fraction of image size - teaches the model to detect partially off-center or partially visible objects.                                                                                    |
| scale        | 0.5               | Random zoom in/out - simulates objects being nearer or farther from the camera.                                                                                                                                                  |
| shear        | 0                 | Random shear (skew) transform - simulates oblique viewing angles.                                                                                                                                                                |
| perspective  | 0                 | Random perspective warp - simulates 3D viewpoint changes.                                                                                                                                                                        |
| flipud       | 0                 | Probability of flipping the image upside-down. Leave at 0 unless upside-down objects are realistic for your use case.                                                                                                            |
| fliplr       | 0.5               | Probability of a left-right mirror flip. Safe and effective for most everyday objects; disable it for tasks where left/right matters (e.g. reading text, left/right-hand tools).                                                 |
| bgr          | 0                 | Probability of swapping the red/blue channels - builds robustness to incorrect channel ordering from some camera pipelines.                                                                                                      |
| mosaic       | 1.0               | Combines 4 training images into one composite, teaching the model to handle varied scene compositions, scales and partial occlusion in a single view. One of the biggest contributors to YOLO's strong small-object performance. |
| mixup        | 0                 | Blends two images and their labels together (semi-transparently) - adds regularization by injecting label noise.                                                                                                                 |
| cutmix       | 0                 | Pastes a rectangular patch from one image onto another, keeping both label sets - builds occlusion robustness.                                                                                                                   |
| copy_paste   | 0                 | Fraction of segmented objects copy-pasted into other images (segmentation/OBB tasks) - especially useful for boosting rare-class frequency.                                                                                      |
| auto_augment | 'randaugme<br>nt' | Predefined augmentation policy (classification task only): randaugment, autoaugment, or augmix.                                                                                                                                  |
| erasing      | 0.4               | Classification only: randomly erases (blacks out) a region of the image so the model can't over-rely on one giveaway feature.                                                                                                    |

Ultralytics also accepts custom Albumentations transforms via the `augmentations` argument (Python API only) for advanced, task-specific pipelines (e.g. simulated rain/fog/motion-blur for outdoor cameras).

**NOTE** — Augmentation should mirror the variation you'll actually see in deployment, not variation for its own sake. A fixed CCTV camera never sees upside-down people, so `flipud=0` makes sense there; the default `fliplr=0.5` is safe almost everywhere since most real-world objects look valid mirrored. Turning every augmentation up to maximum is a common beginner mistake - it can make the training distribution so unrealistic that the model learns to handle noise instead of your actual objects.

## 4.5 Reading the results

Every run writes its outputs (by default) to `runs/detect/<name>/`, including:

| File                             | What to look at                                                                                                                            |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| results.csv / results.png        | Per-epoch loss (box, cls, dfl) and metric (precision, recall, mAP50, mAP50-95) curves for train and validation.                            |
| confusion_matrix.png             | Which classes get confused with which - and how many objects are missed entirely (background row/column).                                  |
| PR_curve.png / F1_curve.png      | Precision-recall trade-off per class and overall, useful for picking a deployment confidence threshold.                                    |
| val_batchN_pred.jpg              | Sample validation images with predicted boxes drawn, for a quick visual sanity check.                                                      |
| weights/best.pt, weights/last.pt | The checkpoint with the best validation metric, and the checkpoint from the final epoch, respectively. Deploy best.pt in almost all cases. |

Healthy training usually looks like: box/cls/dfl loss curves for both train and val trending down and roughly tracking each other, and mAP50-95 climbing and then plateauing. A widening gap between train loss (still falling) and val loss (flat or rising) is the classic overfitting signature described in Part 1.5 - the point to consider more augmentation, more data, a smaller model, or simply stopping earlier.

# Part 5

## Training for Production: Do's and Don'ts

A model that looks great in a Jupyter notebook can still fail badly in production. This section collects the practices that separate a demo model from one that holds up on a live camera feed, a factory line, or a moderation pipeline.

### 5.1 Dataset quality & labeling

#### ✓ DO

- Collect images from the exact conditions the model will run in (same camera type, angle, lighting, resolution, distance) - not just clean stock photos.
- Aim for reasonable class balance, or use weighting/oversampling for rare-but-important classes (e.g. a safety violation that occurs rarely but matters most).
- Have a second person spot-check a sample of labels; label errors silently cap your maximum achievable accuracy.
- Include "hard negatives": images with no target objects, and images with visually similar distractors, so the model learns what NOT to detect.
- Version your dataset (DVC, Roboflow versions, or even a dated zip) so any run can be reproduced exactly.

#### ✗ DON'T

- Don't train only on the 'easy' well-lit, well-centered examples and expect it to handle the messy real world.
- Don't assume more data always helps if the new data is mislabeled or from a totally different distribution - it can hurt.
- Don't ignore edge cases (fog, glare, night, motion blur, partial occlusion) if they'll occur in deployment - the model won't invent robustness you didn't train for.
- Don't mix labeling conventions (e.g. tight vs. loose boxes) inconsistently across the dataset.

### 5.2 Splitting, leakage & evaluation honesty

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>✓ DO</b></p> <ul style="list-style-type: none"> <li>• Split by source (e.g. by video/session/location), not just randomly by frame, when images come from continuous video - adjacent frames are nearly identical.</li> <li>• Hold out a true test set that is touched exactly once, at the end, for the number you report or ship on.</li> <li>• Re-check mAP per class, not just the overall average - one rare class can be near zero while the overall number still looks fine.</li> <li>• Evaluate at the confidence/loU thresholds you'll actually deploy with, not only the default validation settings.</li> </ul> | <p><b>✗ DON'T</b></p> <ul style="list-style-type: none"> <li>• Don't let the same physical scene, object instance, or near-duplicate image appear in both train and val/test (data leakage inflates validation metrics unrealistically).</li> <li>• Don't tune hyperparameters against the test set - that turns it into a second validation set and you lose your honest final number.</li> <li>• Don't trust a single mAP number in isolation - a model can have great mAP and still miss the specific failure mode that matters most for your product.</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.3 Hyperparameters, monitoring & reproducibility

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>✓ DO</b></p> <ul style="list-style-type: none"> <li>• Start from a pretrained checkpoint and sensible defaults; change one thing at a time and compare against a baseline run.</li> <li>• Set a fixed seed and keep deterministic=True for experiments you need to compare fairly.</li> <li>• Watch the loss curves during training (not just the final metric) to catch instability, NaNs, or a bad learning rate early and stop wasted runs.</li> <li>• Use patience (early stopping) and save_period checkpoints so a crashed or overfitting run isn't a total loss.</li> <li>• Log every run's config, dataset version, and metrics somewhere searchable (a spreadsheet is fine at small scale; MLflow/Weights&amp;Biases at team scale).</li> </ul> | <p><b>✗ DON'T</b></p> <ul style="list-style-type: none"> <li>• Don't blindly copy hyperparameters from a different dataset/task without re-validating them on yours.</li> <li>• Don't judge a config from one run - training has run-to-run variance; a couple of repeats gives a much more honest signal.</li> <li>• Don't leave augmentation maxed out through the entire run - close_mosaic (or manually reducing augmentation) in the final epochs, as YOLO11 does by default, helps the model settle on realistic statistics.</li> <li>• Don't train unattended for days without any checkpointing or intermediate validation - hardware and power failures happen.</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.4 Export, deployment & ongoing monitoring

### ✓ DO

- Export to the runtime that matches your deployment hardware (ONNX for portability, TensorRT for NVIDIA GPU latency, OpenVINO for Intel CPU, TFLite/CoreML for mobile) and re-validate accuracy after export - quantization especially can shift results.
- Benchmark actual end-to-end latency (including preprocessing/NMS/postprocessing) on the real target device, not just the training GPU.
- Set a conservative confidence threshold for safety-critical alerts, and a permissive one where missed detections are cheap and false alarms are costly, or vice versa - tune conf per use case, not by default.
- Keep collecting and periodically re-labeling production edge cases ("data flywheel") and retrain - real-world conditions drift over time.
- Version and roll back deployed model weights the same way you would application code.

### ✗ DON'T

- Don't assume validation mAP transfers 1:1 to production - lighting, camera hardware, and object distributions change over time and by site.
- Don't skip a staged rollout (shadow mode / A-B test) before fully trusting a new model version on live decisions.
- Don't ignore silent failure modes - a detector that stops firing (e.g. after a camera firmware update changes color balance) can fail quietly with no error thrown.
- Don't treat a trained model as "done" - object detectors used for surveillance, monitoring or safety need periodic re-evaluation as your environment and edge cases evolve.

# Part 6

## Study Path & Recommended Videos

---

### 6.1 Suggested learning order

1. Get comfortable with the neuron/layer/loss/backprop/gradient-descent picture in Part 1 - watch the 3Blue1Brown videos below alongside it.
2. Understand convolution and CNN vocabulary (Part 2.2) - this is the backbone of every vision model you'll touch, YOLO included.
3. Skim RNNs (Part 2.3) for breadth; don't over-invest here unless your next project involves video/sequence data.
4. Learn the object-detection vocabulary - IoU, NMS, mAP (Part 3.2) - before touching YOLO's code, so the metrics in your results.csv make sense from day one.
5. Install Ultralytics and train YOLO11 on a small public dataset (COCO8, or a Roboflow Universe dataset) end-to-end before bringing in your own data.
6. Read through Part 4.3-4.4 with the docs open, and deliberately change one or two arguments (e.g. lr0, epochs, flipr) to see the effect on your own results.csv.
7. Once a model trains cleanly, read Part 5 before calling anything 'production-ready.'

### 6.2 Video resources

#### Neural networks & backpropagation — 3Blue1Brown

Grant Sanderson's "Neural Networks" series is widely considered the best visual introduction available. Four videos, watch in order:

### 3Blue1Brown - Neural Networks playlist

#### ■ Chapter 1 — But what is a neural network?

<https://www.youtube.com/watch?v=aircAruvnKk>

Neurons, layers, weights, biases - built around recognizing handwritten digits.

#### ■ Chapter 2 — Gradient descent, how neural networks learn

<https://www.youtube.com/watch?v=IHZwWFHwa-w>

The cost/loss function and how gradient descent minimizes it.

#### ■ Chapter 3 — What is backpropagation really doing?

<https://www.youtube.com/watch?v=llg3gGewQ5U>

The intuition behind the backward pass, before the calculus.

#### ■ Chapter 4 — Backpropagation calculus

<https://www.youtube.com/watch?v=tIeHLnjs5U8>

The chain-rule mechanics underneath chapter 3, for anyone comfortable with calculus.

## Convolutions & CNNs

### Watch

#### ■ But what is a convolution?

<https://www.youtube.com/watch?v=KuXjwB4LzSA>

3Blue1Brown. Explains the sliding-kernel operation itself (the math CNNs are built on), using image blurring and edge detection as running examples.

## YOLO & object detection

### Official, version-current sources

#### ■ Ultralytics YOLO documentation & embedded walkthroughs

<https://docs.ultralytics.com/modes/train/>

The official Train guide, which embeds a short "How to train a YOLO model on your custom dataset in Google Colab" walkthrough video and stays current as YOLO versions change.

#### ■ Ultralytics YouTube channel

<https://www.youtube.com/@Ultralytics>

Official channel with dataset-specific training walkthroughs, hyperparameter-tuning guides, and release videos for each new YOLO version.

**NOTE** — 3Blue1Brown does not currently have a dedicated YOLO video - his channel focuses on the underlying math (neural nets, convolutions, linear algebra, calculus), which is exactly what Parts 1-2 of this guide lean on it for. For YOLO specifically, the Ultralytics documentation and channel are the most reliable and up-to-date source, since they're maintained by the team that builds the model and change whenever the API does.

## 6.3 Glossary

| Term                                | Meaning                                                                                                                                                                 |
|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backbone                            | The early part of a detection network (usually a plain CNN) that extracts general visual features from the raw image.                                                   |
| Neck                                | The middle part of a detector that combines features from different backbone depths/scales (e.g. FPN/PAN-style layers) before the head makes predictions.               |
| Head                                | The final part of the network that turns processed features into actual outputs - box coordinates, objectness, class scores.                                            |
| Checkpoint                          | A saved snapshot of a model's weights at a point in training (e.g. best.pt, last.pt).                                                                                   |
| Fine-tuning                         | Continuing to train a pretrained model on new, usually smaller, task-specific data instead of training from scratch.                                                    |
| Transfer learning                   | The general strategy of reusing a model trained on one task/dataset as a starting point for a related task - fine-tuning is one way of doing it.                        |
| Inference                           | Running a trained model on new data to get predictions, as opposed to training.                                                                                         |
| Confidence threshold                | The minimum objectness/class score a prediction must have to be kept, applied at inference time (the conf argument).                                                    |
| ONNX / TensorRT / OpenVINO / TFLite | Portable or hardware-specific formats a trained model can be exported to for deployment outside the original training framework.                                        |
| Quantization                        | Reducing the numeric precision of a model's weights (e.g. 32-bit to 8-bit integers) to shrink its size and speed up inference, usually with a small accuracy trade-off. |

---

*Parameter tables and defaults in Part 4 are drawn from the official Ultralytics configuration documentation ([docs.ultralytics.com/usage/cfg](https://docs.ultralytics.com/usage/cfg)) at the time of writing and may shift slightly across library versions - always cross-check against the installed ultralytics package's own docs/--help output before a production run.*